

RAG (Retrieval-Augmented Generation) in AI Notebook Project

1. What is RAG?

RAG stands for Retrieval-Augmented Generation. It is an architectural technique where a Large Language Model (LLM) is combined with an external retrieval system. Instead of relying only on its internal training data, the LLM first retrieves relevant information from user-provided documents and then generates an answer based on that information.

In simple terms, RAG allows the LLM to read your notes or documents before answering your question.

2. Why RAG is Needed in This Project

- 1 LLMs do not know the user's personal notes or documents.
- 2 Training an LLM on new data is expensive and impractical.
- 3 Without RAG, LLMs may hallucinate or give generic answers.
- 4 RAG enables accurate, document-based answers without retraining.

3. Project Technology Stack (RAG Only)

- 1 LLM: LLaMA-3-8B-Instruct (for answer generation)
- 2 Embedding Model: BAAI/bge-small-en-v1.5 (for vector representation)
- 3 Vector Database: ChromaDB (for similarity search)
- 4 Document Processing: PyPDF2, python-docx, Tesseract OCR, Pillow

4. How RAG is Implemented in This Project

- 1 User uploads documents such as PDFs, DOCX files, or images.
- 2 Text is extracted from documents using appropriate tools.
- 3 Extracted text is divided into fixed-size chunks.
- 4 Each text chunk is converted into embeddings using the BGE model.
- 5 Embeddings are stored in ChromaDB along with metadata.
- 6 User queries are converted into embeddings.
- 7 ChromaDB retrieves the most relevant text chunks.
- 8 Retrieved chunks are passed to the LLaMA-3-8B model.
- 9 The LLM generates an answer grounded in retrieved content.

5. Step-by-Step Process to Start the Project (Without ML)

- 1 Set up MERN stack for frontend and backend services.
- 2 Create a Python FastAPI service for AI processing.
- 3 Implement document text extraction pipelines.

- 4 Apply simple fixed-size chunking strategy.
- 5 Generate embeddings using the embedding model.
- 6 Store and query embeddings using ChromaDB.
- 7 Connect query retrieval with LLaMA-3-8B-Instruct.
- 8 Return AI-generated answers to the frontend.

6. Conclusion

This project uses RAG to provide accurate and context-aware answers from user-uploaded documents. By implementing RAG without additional machine learning models, the system remains simple, efficient, and suitable as a strong foundational AI notebook application.